

Hochschule Karlsruhe
University of
Applied Sciences



Patching as a Service Linux Server

Thomas Merz | 2025





dm in Deutschland und Europa

KURZPORTRAIT dm



Führender Drogeriemarkt
u.a. in Deutschland



2.131 Filialen in Deutschland
4.116 Filialen in Europa



Marktpräsenz
in 14 Ländern Europas



60.373 Mitarbeitende in Deutschland
89.197 Mitarbeitende im dm-Konzern



12,47 Mrd. Euro Umsatz im GJ 2023/24
17,74 Mrd. Euro Umsatz im dm-Konzern



dm IN DEUTSCHLAND



60.373

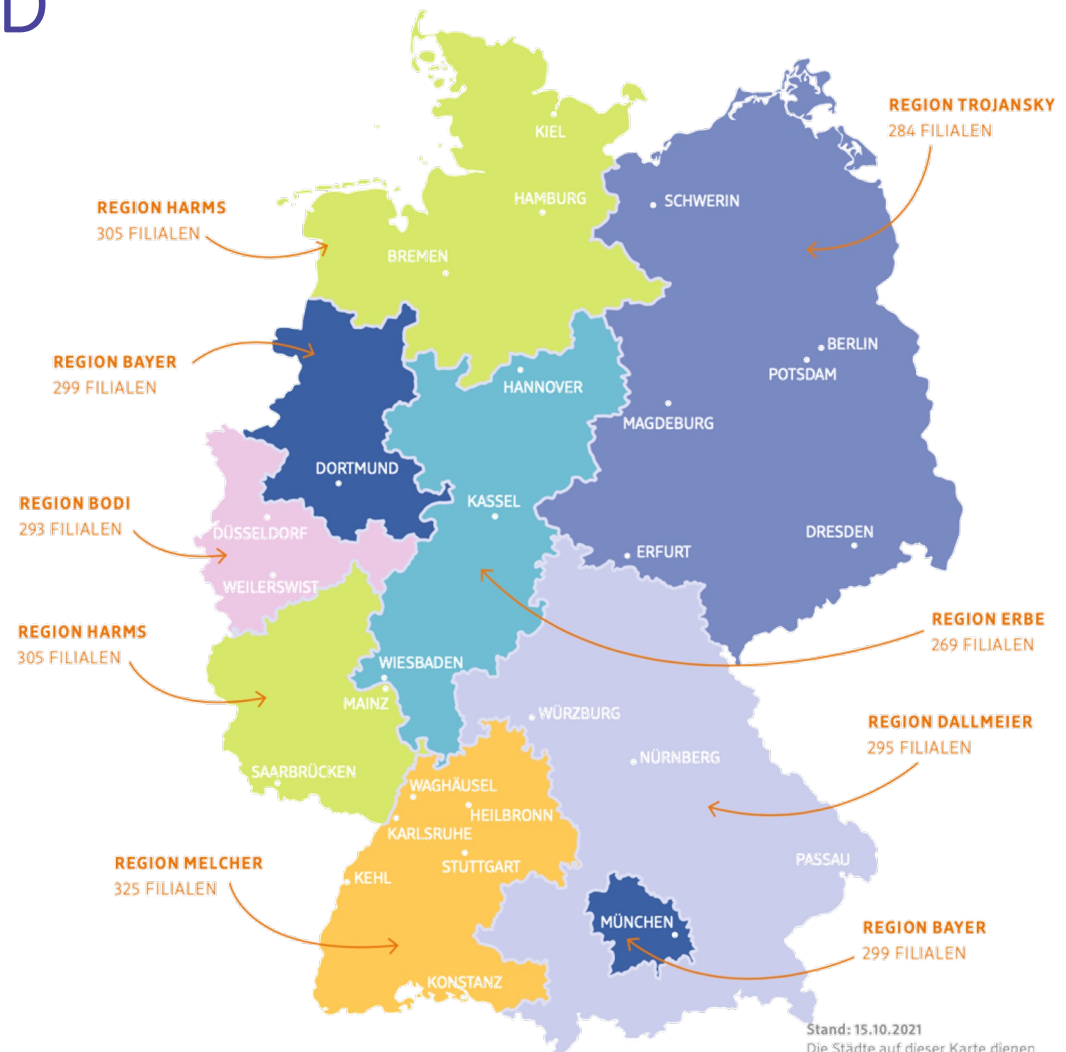
Mitarbeitende

2.131

dm-Märkte von Süderlügum
bis Sonthofen

12,47

Milliarden Euro Umsatz im
Geschäftsjahr 2023/24



Stand: 15.10.2021
Die Städte auf dieser Karte dienen
der Orientierung und sind nicht immer
identisch mit Filialstandorten.

dm IN EUROPA



28.824

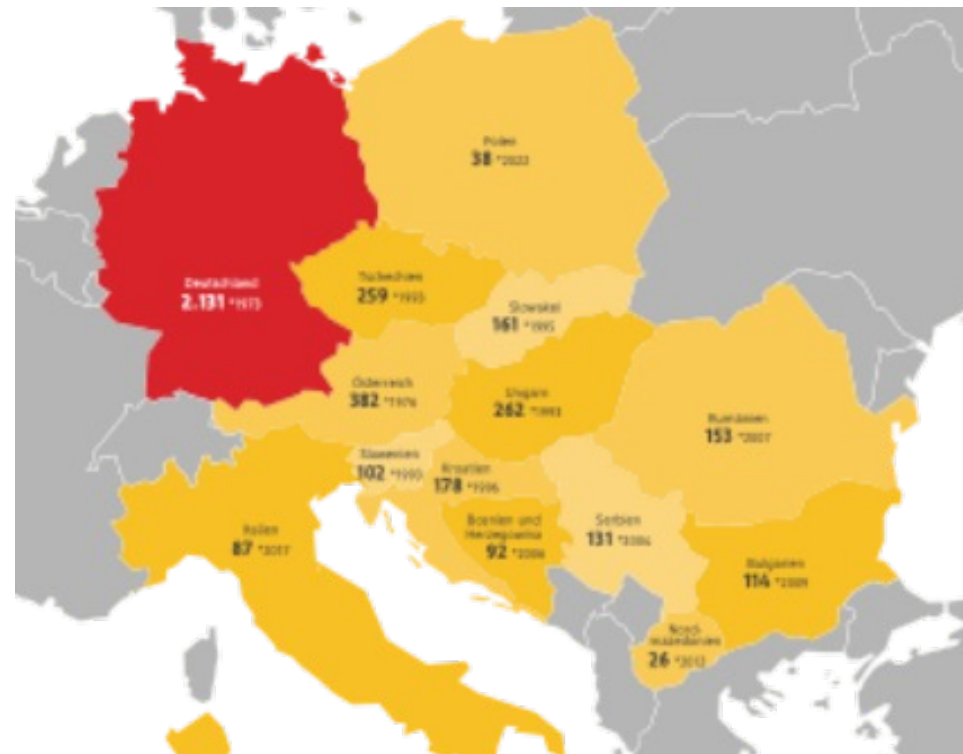
Mitarbeitende

14

Vierzehn Verbundene Länder
in Europa

5,25

Euro Umsatz im Geschäftsjahr
2023/24



Stand 30.09.2024
*Eröffnungsjahr

KURZPORTRAIT dmTECH



100%-ige Tochterfirma
der dm-drogerie markt GmbH + Co. KG



Gegründet 1988
als „FILIADATA GmbH“ & 2018 umfirmiert
in „dmTECH GmbH“



gesamte IT des dm-Konzerns
für 14 Länder liegt in unserer Verantwortung



Über 1300 Mitarbeiter
davon rund 80 Auszubildende und duale
Studierende



Sitz in Karlsruhe
seit 2019 im dm-dialogicum in Durlach



WHOAMI



- **Linux seit 1994 (zwischen durch mal was mit Großrechnern, DB2 und SAP)**

- seit 1991 Linux

- **>20 Jahre bei dm(TECH)**

- davor: Studium Angewandte Informatik / Telekommunikation + WEB.DE

- **Bereitstellung von Infrastruktur für das**

- Deployment/Lifecycle
- Config-Management
- Patch-Management
- Monitoring
- von Linux Servern (RHEL und SLES; VMs und Hardware)

- **Open Source Fan**



Herausforderungen und wie man sie meistert...

PATCHING LINUX SERVERS



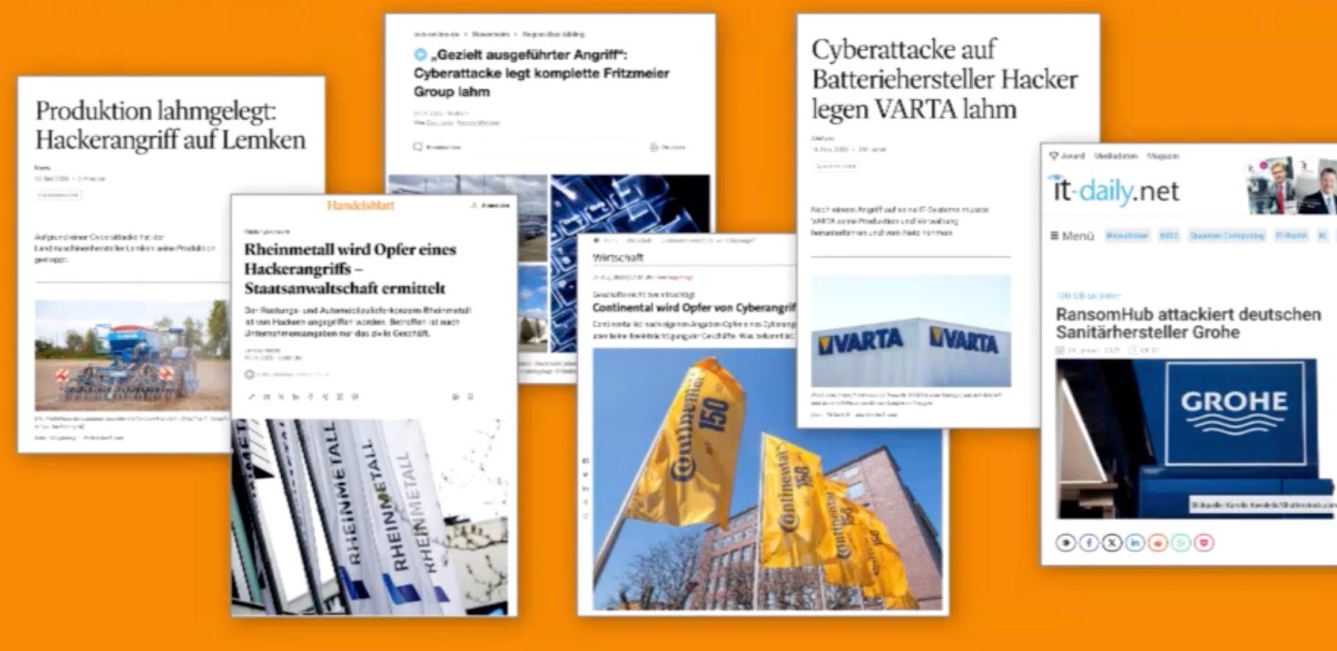
„State of OT Security Report 2024“ von Palo Alto Networks

„Fast 70% der Industrieunternehmen haben letztes Jahr einen Cyberangriff erlebt.“

Befragung von weltweit 1.979 Führungs- und Fachkräfte aus dem Bereich
OT und IT aus 16 verschiedenen Branchen

Quelle: <https://www.paloaltonetworks.com/resources/research/state-of-ot-security-report>

Wer ist der nächste?



Warum ist Patchen von OT-Systemen so wichtig?

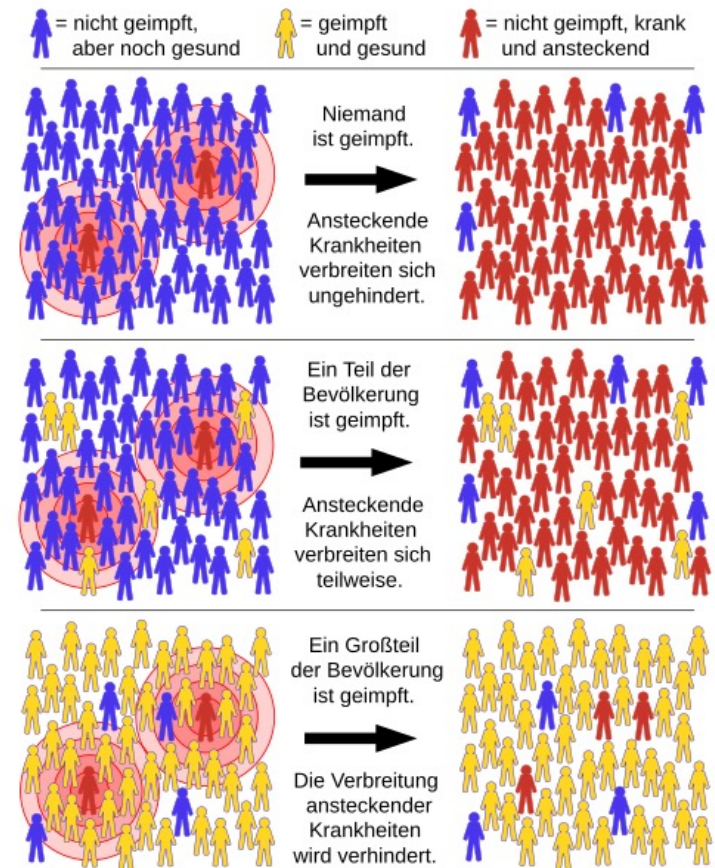
- **Fertigungsindustrie im Fokus von Ransomware**
Teure Ausfallzeiten = Hohe Bereitschaft Lösegelder zu zahlen
- **OT wird zunehmend zu einem strategischen Ziel**
Immer mehr gezielte Attacken auf industrielle Systeme, auch durch staatliche Akteure
- **Compliance und Gesetze verlangen aktives Patch-Management**
Interne Richtlinien, NIS2, CRA, IEC 62443 & Co. machen Patch-Prozesse zur Pflicht
- **Virtuelles Patchen reicht nicht aus**
Firewalls helfen nur bedingt, Innentäter und Memory Leaks bleiben ein Risiko
- **Patchen reduziert das Gesamtrisiko**
Weniger Schwachstellen bedeuten geringere Angriffsfläche

VORAB: FRAGEN ANS PUBLIKUM

- Wer patcht sein(e)/ihr(e) Linux(e) und Docker Container selber?
- Wie oft?
- Wie?
- Rebootet oder restartet ihr auch eure Dienste/Anwendungen?
- Wie viel “Spaß” macht das?

0. WARUM PATCHEN?

- Sicherheit, Stabilität(?) und langfristige Wartbarkeit(!)
 - Es **schützt** nicht nur die **Server** selbst,
 - sondern auch **die darauf gespeicherten Daten**
 - und die **gesamte IT-Infrastruktur**. (sog. "Herdenimmunität")
-
- **Achtung: Impf-/Patch-Müdigkeit vs. Impf-/Patch-Pflicht**



1. Sicherheit

- **Schutz vor Angriffen:** Sicherheitsupdates schließen bekannte Sicherheitslücken, die von Angreifern ausgenutzt werden können.
- **Vertraulichkeit und Integrität:** Durch das Schließen von Schwachstellen wird sichergestellt, dass Daten nicht unbefugt abgerufen oder verändert werden können.

2. Stabilität

- **Fehlerbehebungen:** Updates beheben Bugs und verbessern die Stabilität des Systems, was zu einer besseren Leistung führt.
- **Vermeidung von Abstürzen:** Regelmäßige Patches können dazu beitragen, unerwartete Systemabstürze zu verhindern.

3. Kompatibilität

- **Neue Funktionen:** Patches können neue Funktionen oder Verbesserungen enthalten, die die Funktionalität des Servers erweitern.
- **Softwarekompatibilität:** Updates sorgen dafür, dass der Server mit der neuesten Software und den neuesten Technologien kompatibel bleibt.

4. Support und Wartung

- **Langfristiger Support:** Viele Linux-Distributionen bieten nur für eine bestimmte Zeit Sicherheitsupdates an. Ein veraltetes System erhält möglicherweise keine Unterstützung mehr.
- **Community und Dokumentation:** Aktuelle Systeme profitieren von aktiverer Unterstützung durch die Community und bessere Dokumentation.

5. Compliance

- **Einhaltung von Vorschriften:** In vielen Branchen sind regelmäßige Updates und Patches erforderlich, um gesetzlichen Anforderungen und Standards zu entsprechen.

1. WANN PATCHEN?

“As soon as ASAP?” 🙄

Wöchentlich?

Monatlich?

Quartalsweise?

Aus Verzweiflung: Gar nicht? 😞

BSI gibt Hinweise:



Bundesamt
für Sicherheit in der
Informationstechnik

OPS.1.1.3 Patch- und Änderungsmanagement

“OPS.1.1.3.A15 Regelmäßige Aktualisierung von IT-Systemen und Software (B)

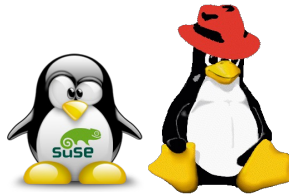
IT-Systeme und Software SOLLTEN **regelmäßig** aktualisiert werden.

Grundsätzlich SOLLTEN Patches **zeitnah** nach Veröffentlichung eingespielt werden. Basierend auf dem Konzept für das Patch- und Änderungsmanagement MÜSSEN Patches **zeitnah** nach Veröffentlichung bewertet und entsprechend **priorisiert** werden. Für die Bewertung SOLLTE geprüft werden, ob es zu diesem Patch bekannte Schwachstellen gibt. Es MUSS entschieden werden, ob der Patch eingespielt werden soll. Wenn ein Patch eingespielt wird, SOLLTE kontrolliert werden, ob dieser auf allen relevanten Systemen zeitnah erfolgreich eingespielt wurde. Wenn ein Patch nicht eingespielt wird, MÜSSEN die Entscheidung und die Gründe dafür dokumentiert werden”

2. VERSCHIEDENE LINUX U. ANWENDUNGEN

Distributionen

- RHEL
- SLES



Major versions

- 7, 8 und 9 + EUS
- 12 und 15 + SP

Paketmanager

- yum, dnf
- zypper

Ca. 90% VMs und 10% HW

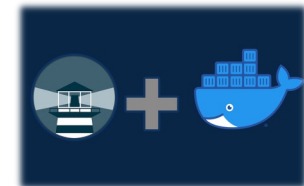
Peak: ca. 5000 Server

Anwendungen!

- SAP
- DWH
- Hadoop
- Datenbanken...
- Materialflussrechner
- Druckserver
- SFTP
- ELK
- Backupserver
- Quorumnodes
- OTP für VPN
- Eigene Tools
- Infrastruktur für Betrieb der Linux Server

3. ANWENDUNGEN IM DOCKER CONTAINER

- **Basis-Images regelmäßig aktualisieren → docker pull/build**
 - offizielle Images aus Docker Hub, die regelmäßig aktualisiert werden
 - latest-Tag vs. Tag/Version gezielt im Dockerfile anpassen/erhöhen
 - apt update && apt-upgrade im Dockerfile
 - Automatisierung mit CI/CD → GitHub Actions, GitLab CI oder Jenkins
- **Watchtower**
 - überwacht und aktualisiert Docker-Container
 - kann als Container ausgeführt werden und prüft regelmäßig, ob Updates verfügbar sind
- **Portainer**
 - webbasierte Benutzeroberfläche für die Verwaltung von Docker-Containern
 - einfache Verwaltung von Containern und Images,
 - einschließlich der Aktualisierung und Überwachung
- Skripte (“docker pull”) und Cronjobs/SystemD-Timer...



3. ANWENDUNGEN IM DOCKER CONTAINER

■ Sicherheitsüberprüfungen

- **Trivy** (Open-Source-Sicherheitsscanner, überprüft Container-Images auf bekannte Sicherheitsanfälligkeiten)
- **Clair** (Open-Source-Tool zur statischen Analyse von Container-Images, das Sicherheitsanfälligkeiten identifiziert)
- **Snyk** (Tool zur Sicherheitsüberprüfung von Container-Images, bietet auch Abhängigkeitsmanagement)

■ Container-Rollouts

- Rolling Updates (Minimierung/Vermeidung von Ausfallzeiten)
- zuerst “Test” und dann “Produktion”!

KANN MAN SICH DEN GANZEN AUFWAND,
DIE VIELEN BEWERTUNGEN UND
ENTSCHEIDUNGEN NICHT AUCH SPAREN???

TL;DR: Ja!

SOLLEN... MÜSSEN... MACHEN!!!

Machen ist wie wollen, nur krasser.

- Durch Automatisierung und mit Leidenschaft ❤️ machen wir das
 - trotz vieler unterschiedlicher Systeme
 - und vieler unterschiedlicher Anforderungen
 - besser und vor allem flexibler als “die Cloud”

TROTZDEM NOCH HERAUSFORDERUNGEN...

- **Wichtige** (business-critical wie z.B. SAP, produktiv, 100 Entwickler/Externe, ...)
- Und **weniger wichtige** Systeme / Stages
(Sandbox, Test, Dev, QA, Konsolidierung, Pre-Prod, ...)
- **Alle Patche sollten vor den wichtigen Systemen getestet werden!**
 - D.h. nicht alles auf einmal patchen und
 - immer denselben Stand in allen Stages innerhalb einer Landschaft ausrollen...
- **“Ich traue mich nicht” oder “ich kann das doch gar nicht und weiss gar nicht wie das geht”...**
 - Anwendungsverantwortlicher ist auch für das Patchen des OS zuständig!
 - zuständig <> tun, d.h. kümmern/delegieren ist auch “ok”!

WIE HAT **DMTECH** DAS DILEMMA GELÖST?

■ Bottom-Up

- Alle Linux Server werden gepatcht! (Opt-Out) - sonst:
 - Manuell
 - Ansible
 - Cronjob
 - Andere Automatisierungs-Tools...
- Bereitstellung von
 - Infrastruktur
 - Hilfsmittel(n)

Wir prüfen, DASS gepatcht wird!!!

INFRASTRUKTUR-UPGRADE

■ zentraler Reposerver (kein Server hat direkten Zugang zum Internet)

- RHEL
- SLES
- EPEL
- weitere Hersteller, OpenSource, ...



■ Staging

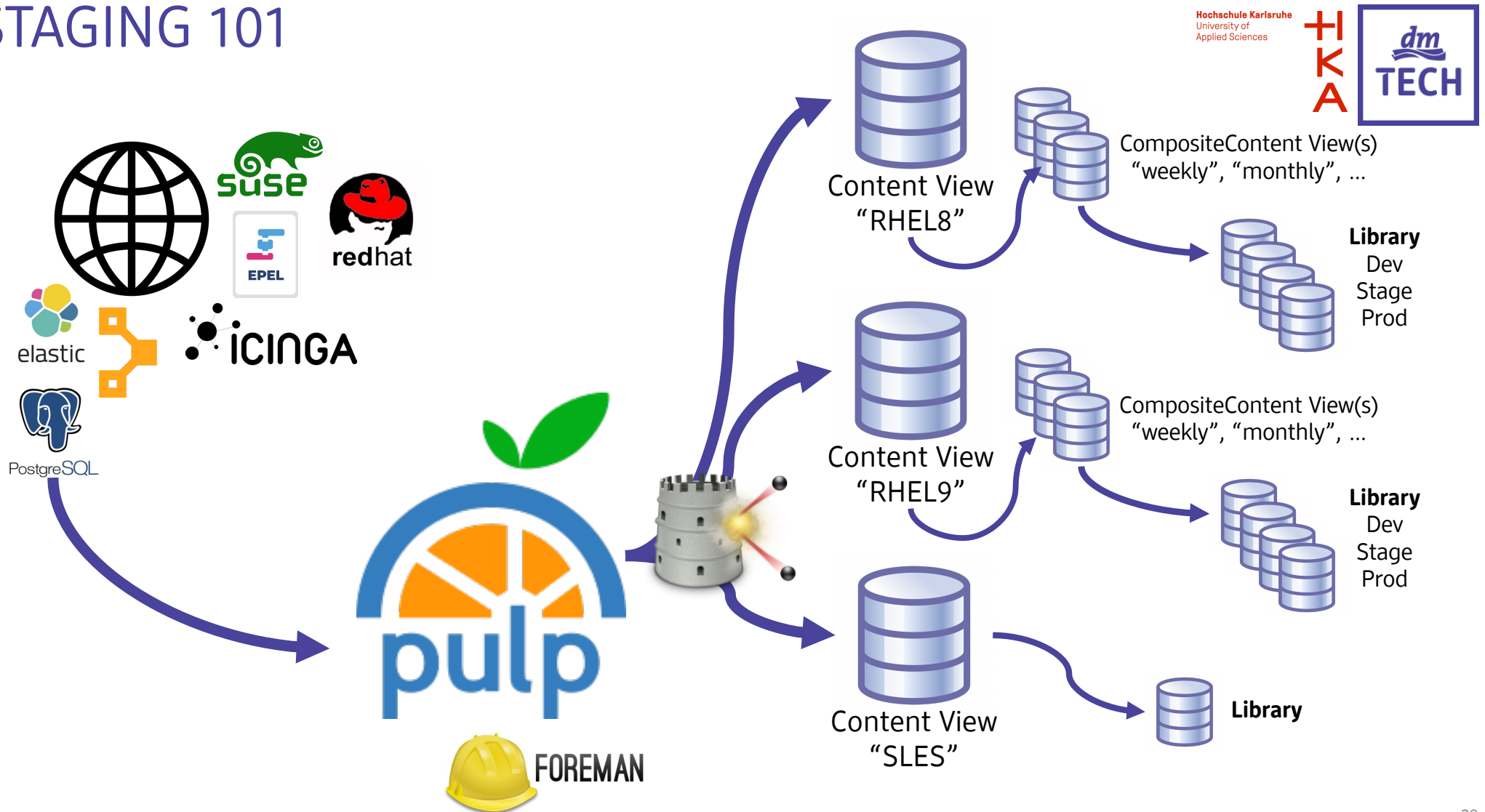
- Library, dev, stage, prod → Abbildung von 1-n Stages auf “zuerst/vorab, dann alles nonprod, zuletzt prod”
- Verschiedene Zyklen je Anwendung: wöchentlich, monatlich, ad-hoc

Content views

4 Content views
22 Composite content views

Version	Environments	Packages	Errata	Additional content
☐ Version 235.0	dev 3 days ago Library 3 days ago stage 1 day ago	83643	10154 (2167, 4073, 1361)	1150 Module streams
☐ Version 234.0	prod 7 days ago	83219	10132 (2150, 4071, 1359)	1141 Module streams

STAGING 101



INFRASTRUKTUR-UPGRADE

- **Zuweisung von Systemen/Gruppen zu Zyklen/Stages**
 - Anhand “Activation Key” in Puppet-Hieradata



```
data/app_tier/lxstage.yaml
3:service_repositories::rhel::katello: activationkey: w

data/app_tier/lxperf.yaml
3:service_repositories::rhel::katello::activationkey: w

data/app_tier/lxrls.yaml
3:service_repositories::rhel::katello::activationkey: w

data/app_tier/lxsbx.yaml
7:service_repositories::rhel::katello::activationkey: w

data/app_tier/lxtst.yaml
7:service_repositories::rhel::katello::activationkey: w

data/app_tier/lxdev.yaml
5:service_repositories::rhel::katello::activationkey: w

data/app_tier/lxprod.yaml
12:service_repositories::rhel::katello::activationkey: w

data/common.yaml
470:service_repositories::rhel::katello::activationkey: w
```



Dev = w_di
Stage = w_mi
...
Prod = w_mo

PUPPET-HIERA (N-DIMENSIONAL):



```
hierarchy:
- name: Node
  paths:
  - node/%{trusted.certname}.yaml
  - hostgroup/%{hostgroup}.yaml
  - owner_name/%{owner_name}.yaml
- name: Project
  paths:
  - project/%{facts.project}/%{facts.system_type}/%{facts.app_tier}.yaml
  - project/%{facts.project}/%{facts.system_type}/common.yaml
  - project/%{facts.project}/%{facts.app_tier}.yaml
  - project/%{facts.project}/common.yaml
- name: Location and Datacenter
  paths:
  - location/%{location}/%{facts.app_tier}.yaml
  - location/%{location}/RZ%{facts.datacenter}.yaml
  - location/%{location}.yaml
- name: App-Tier
  paths:
  - app_tier/%{facts.app_tier}.yaml
- name: Common
  path: common.yaml
- name: Core-Modules
  paths:
  - "../core_modules/data/location/%{location}/%{facts.app_tier}.yaml"
  - "../core_modules/data/location/%{location}/RZ%{facts.datacenter}.yaml"
  - "../core_modules/data/location/%{location}.yaml"
  - "../core_modules/data/app_tier/%{facts.app_tier}.yaml"
  - "../core_modules/data/common.yaml"
```


PUPPET-HIERA: PATCHING-ZEITEN, OPT-"IN", ...



```
# Start the Updatescript at every day listed
service_updatesystem::days:
- Mon
- Tue
- Wed
- Thu
# Start the Updatescript once for every hour listed
# Do NOT use a leading 0 for numbers <9
service_updatesystem::enabled: true
service_updatesystem::foreman_dlm_url: https://foreman
service_updatesystem::hours:
- 2
- 3
- 4
service_updatesystem::packages: []
service_updatesystem::service_provider: init
# RedHat only
service_updatesystem::services_to_restart: []
service_updatesystem::unwanted_packages: []
```

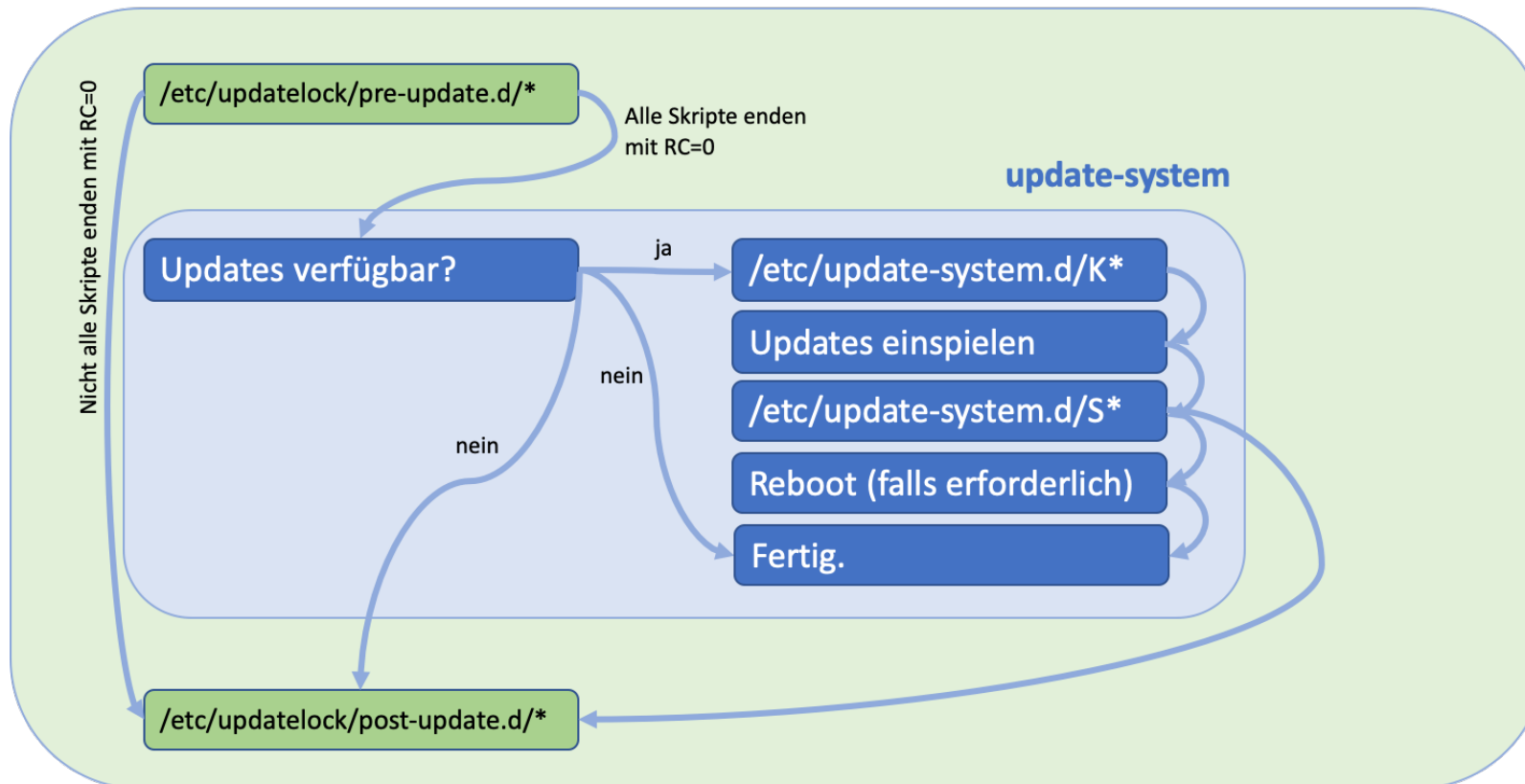
HILFSMITTEL:

BASH, SYSTEMD, CRON, ANSIBLE, ...

- **All-in-One “update-system” bash Skript mit “init.d”-style Hooks**
 - RHEL, SLES (erweiterbar um z.B. Debian, ...)
 - yum, dnf, zypper
 - 1210 loc, 1297 inkl. #, 27 functions, davon 2 traps
- **SystemD-Timer (Default)**
 - Feste “zufällige” Startzeit MO-DO zw. 2 und 5
 - Bei 2er (oder mehr) Systemverbünden nie mehr als ein System gleichzeitig (Locking per Foreman-Plugin “DLM”)
- **Cronjobs (per Puppet oder manuell eingetragen) statt SystemD-Timer**
- **Manuell oder Ansible Playbooks (manuell oder per GitLab Pipeline)**
- ...

HILFSMITTEL: DAS HERZSTÜCK!

„Auto-Patching“ (updatelock-update.service)



UPDATE-SYSTEM FUNKTIONEN

- `check_space_needed()`
- `cleanup_logfiles()`
- `cleanup_old_kernels()`
- `facts_only()`
- `get_additional_repos()`
- `is_reboot_necessary()`
- `is_updates_available()`
- `is_updates_available_rhel()`
- `is_updates_available_sles()`
- `is_updatesystem_globally_enabled()`
- `lock()`
- `manage_repos()`
- `reboot()`
- `restart_specified_services()`
- `run_hooks()`
- `trap_on_EXIT()`
- `trap_on_KILL()`
- `update_agents_rhel()`
- `update_facts()`
- `update_facts_rhel()`
- `update_rhel()`
- `update_sles()`
- `update_system()`
- `usage()`
- `with_puppet()`
- `with_reboot()`
- `with_secpatches_only()`

UPDATE-SYSTEM ABLAUF

- Disable/enable Repos
- **Clean up old Kernels**
- Check for sufficient Space
- Evaluating global Switch
- Deleting logfiles
- Run update just for some agents
- **Updates available? y/n**
- Check for eventually running Puppet + disable Puppet

UPDATE-SYSTEM ABLAUF (2)

- K-Hooks
- Patchen!
- S-Hooks
- Search for `.rpmnew`, `.rpmsave` and `.rpmorigfiles` which may need your review
- Checking and **restarting specific services** to prevent unnecessary reboot
- Stats
 - Number of Patches available + Number of Security-Patches available from all Repos
- **Enable + run Puppet**
- write some (more) statistics for this run
- **Reboot?**

LESSONS-LEARNED SEIT 2015

- Allow for german lang setting
- Fixed a problem that all systems won't be rebooted via auto-patching
- Fix missing Option
- fix needsrestart-check in update_security.sh
- fix order of checks
- fix time for hanging puppet-processes
- Fix Shutdown of Automation
- fix 'check for (sec)updates'
- fix dependencies
- fix 'failed to disable puppet'
- fix updatelock for non-de-domains
- add same fix for /boot
- 🚚 fix 'remove old kernels' for RHEL8
- 🚚 hotfix for RHEL8 due to larger initramfs/kernel needings
- 🐛 fix 'check if automation-agent is installed'
- ⚠️ fix forced-reboot on project xyz
- fix force reboot due too high uptime
- fix osmajors for 6 and 7
- fix rescue block for puppet 4
- fix Returncode of Puppet
- fix update-system for RHEL 7.8 migration
- update_rhel7x_to_rhel74.sh: fix quiet

HILFSMITTEL: AGENTEN

■ “Automation-Workflow” (SAP und DWH)

- Stop Verarbeitung
- Stop Anwendung
- Vmware-Snapshot erstellen (per Foreman-API)
- **Patchen inkl. Reboot**
- Vmware-Snapshot löschen
- Start Anwendung
- Start Verarbeitung

(betrifft 1-n Linux Server wie z.B. DB-Cluster über n VMs)

(“pre-update.d” / “K-Hooks”)

(“pre-update.d” / “K-Hooks”)

(“post-update.d” / “S-Hooks”)

(“post-update.d” / “S-Hooks”)

PATCHING-INFOS... EVERYWHERE!

```
Welcome on [redacted] (GNU/Linux 4.18.0-553.37.1.el8_10.x86_64 Linux)
System Owner (Ali): [redacted]
System uptime: 8 days
For pending (security) updates or pending reboots for this host please check:
https://foreman.\[redacted\]
The system owner is solely responsible for this host!
Last successful update-system : 2025-02-18T04:00:57+01:00
update-system is failing since: 2025-02-19T02:00:31+01:00
Issue tracker: https://\[redacted\]
On-call service: +49 721 [redacted]
Puppet-Server: [redacted]
Puppet-Environment: [redacted]
OS-Release: CAPRI+TECH Red Hat Enterprise Linux 8.10 (Ootpa)
[sshrhc] Welcome [redacted] You are logged in with sshrc.
```

>_
SSH

Patch-Status: failed

Patch-Status: failed

851

Services

5

846

Patch-Status: Security available

Patch-Status: Security available

851

Services

15

5

831

Patch-Status: Security available (LX)

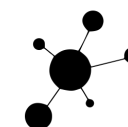
Patch-Status: Security available (LX)

75

Services

1

74



Häufigste Fehlerquellen:

#1 **Filesysteme** zu klein!!!

#2 **Dependencies** bei manuell (nicht aus den bereitgestellten Repos) installierten Paketen!

→ Hooks zum Vor-/Nachbereiten, Extra-Repos möglich, muss man aber auch explizit nutzen...

„MONITORING“ DES PATCHENS



- ✓ `facts.update_system_last_success_actioncount != null`
 - zeigt Dir an, welche Systeme **heute oder beim letzten Patching** Updates bekommen haben.
- ✓ `fact=update_system_last_success_actioncount and value != null`
 - zeigt Dir an, welche Deiner Systeme **heute oder beim letzten Patching wie viele Updates** bekommen haben.
- ⚠ `fact=update_system_updatecount and value > 0`
 - zeigt Dir an, welche Deiner Systeme **noch Patches ausstehend** haben, **die nicht vom Auto-Patching/update-system abgedeckt werden**. Ursache dafür ist i.d.R., dass die Patches aus Repos kommen, die nicht inkludiert werden. Siehe "[Kann ich über Hieras zusätzliche Repos definieren?](#)"
- ✗ `fact = update_system_secupdatecount and value > 0`
 - zeigt Dir an, welche Deiner Systeme **noch Security-Patches ausstehend** haben, **die nicht vom Auto-Patching/update-system abgedeckt werden**. Ursache dafür ist i.d.R., dass die Patches aus Repos kommen, die nicht inkludiert werden. Siehe "[Kann ich über Hieras zusätzliche Repos definieren?](#)"
- ✗ `facts.update_system_failing_since != never`
 - zeigt Dir an, welche Deiner Hosts und wie lange schon **Probleme beim Patching** haben! Wenn Du Dir nicht selbst zu helfen weisst, kannst Du ein Jira-Ticket bei uns aufmachen und wir kümmern uns darum.
- ✗ `errata_status = security_needed and facts.update_system_secupdatecount = 0`
 - zeigt Dir an, welche Deiner Hosts Du unbedingt mal **manuell überprüfen und mit Sicherheitsupdates versorgen** solltest! Für diese Hosts sind sehr wahrscheinlich Pakete ge-versionlock-t und müssen daher manuell aktualisiert werden. Wenn Du dabei Hilfe brauchst, kannst Du gerne ein Jira-Ticket bei uns aufmachen und wir helfen Dir dabei oder kümmern uns darum.

„MONITORING“ DES (NICHT-)PATCHENS ODER IRGENDWAS-VERGESSENS

⚠ `errata_status = errata_needed` ("orange" wegen irgendwelcher ausstehenden Updates üblicherweise)

✖ `errata_status = security_needed` ("rot" wegen ausstehender Sicherheitsupdates)

⚠ `trace_status = process_restart_needed` ("orange" weil irgendein Prozess oder Dienst restartet werden muss)

✖ `trace_status = reboot_needed` ("rot" wegen "Reboot needed")



- **Kenne Deine Anwendungen/Kunden/Systeme!**

- Was lässt sich wann und wie (SystemD-Timer, Cronjob) patchen?
- Wer patch lieber selber und wie (manuell, Ansible, “Agent”)?

- **Stelle Tools und Infrastruktur bereit!**

- Fürs “Patching as a Service”-rundum-sorglos-Paket
- Fürs Patchen allgemein, für Minor-Version-Updates (EUS bei RHEL und ServicePacks bei SLES)
 - Puppet-Hiera bei RHEL
 - Skript bei SLES

- **Aufwand und Kosten für sehr unwahrscheinlichen Ausfall nach Patchen**

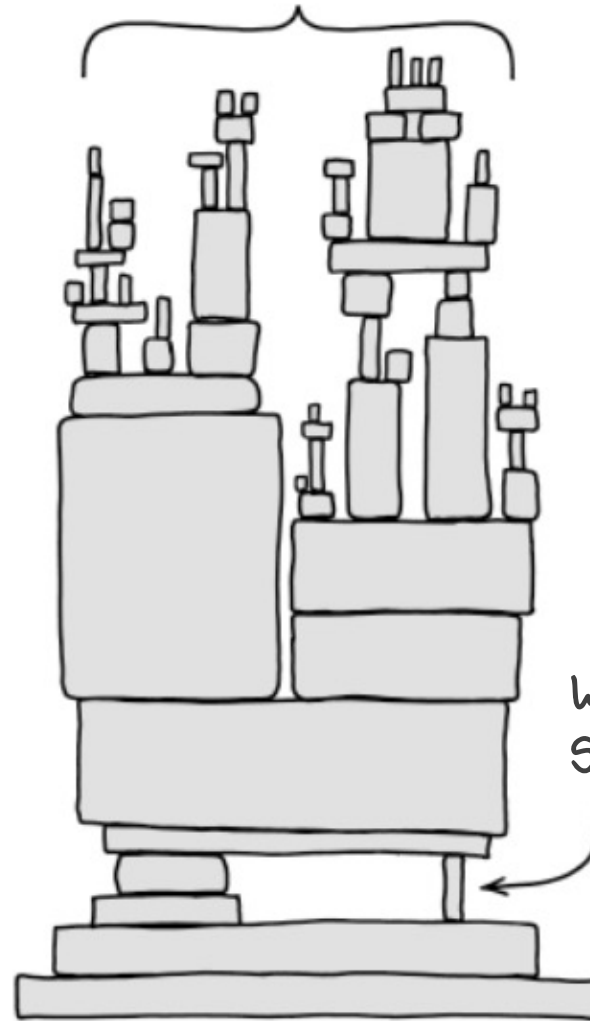
sind sehr viel geringer als für

- einen Sicherheitsvorfall oder gar eines erfolgreichen Angriffs
- bzw. manuelles Patchen

EIN PAAR STATS ZUM ENDE

- All-In-100%-Autopatching: 30% ✓
- Patching manuell/Ansible: 29% ➡
- Patching automatisch per GitLab-Ansible-Pipeline: 4% ➡
- Patching automatisch per “Agent”: 37% ➡
- Systeme, die “update-system” nicht benutzen: 0% ✓
- Systeme, die vollautomatisch gepatcht werden: 71% ✓
- Laufender Aufwand:
 - solange das Monitoring ruhig bleibt: KEIN Aufwand
 - sonst: ab und zu mal ein Filesystem (bzw. prophylaktisch auf mehreren Servern) vergrößern

ALL ENTERPRISE IT INFRASTRUCTURE



WERKS-
STUDENTINNEN



Alle offenen Stellen unter dmtech.tech/de/job-listing





Schau auch gerne bei
uns vorbei.



dmTECH.tech



dmTECH Deutschland



dmtech_deutschland



Kontakt:
dmTECH@dm.de

Vielen Dank.